

Architectural and Algorithmic Blueprint for High-Fidelity Satellite Imagery Super-Resolution

The challenge of bridging the resolution gap in satellite imagery represents a critical frontier in modern remote sensing, computational vision, and geospatial analysis.¹ Publicly available satellite constellations, such as the European Space Agency's Sentinel-2, provide high-cadence, multispectral imagery at a spatial resolution of 10 meters per pixel. While invaluable for macro-level environmental monitoring, agricultural assessment, and regional climate tracking, this spatial resolution remains fundamentally insufficient for fine-grained urban analysis. At 10 meters per pixel, critical objects such as small structures, vehicles, individual trees, and road lane markings remain visually indistinguishable, appearing only as blurred sub-pixel artifacts.¹ Conversely, commercial satellite platforms (such as the Airbus SPOT 6/7 or Maxar WorldView constellations) capture sub-meter to 1.5-meter high-resolution imagery, delivering exceptional clarity. However, acquiring this commercial data is financially prohibitive for large-scale, continuous global monitoring, creating an accessibility barrier for researchers and humanitarian organizations.¹

This comprehensive report outlines an exhaustive, industry-grade algorithmic pipeline and architectural guide for developing a deep learning model capable of executing 4x or 8x super-resolution on Sentinel-2 imagery. The primary objective is to construct a highly advanced system that synthetically generates commercial-grade high-resolution outputs from low-resolution inputs without introducing fabricated geospatial features. The subsequent sections detail the precise logical steps required to build this cutting-edge pipeline, strictly adhering to an architectural blueprint rather than relying on fundamental, pre-written code snippets. The methodology encompasses local-to-cloud infrastructure synchronization, dynamic geospatial data streaming, advanced vision transformer modeling, aggressive out-of-core memory management, and interactive validation systems.

Local IDE to Cloud Infrastructure Synchronization Workflow

Developing robust machine learning pipelines necessitates a hybrid engineering approach. Local Integrated Development Environments (IDEs), such as the Antigravity IDE, are utilized for rapid prototyping, debugging, syntax validation, and unit testing. However, the computational demands of training a super-resolution vision transformer require dedicated hardware accelerators. Transitioning a project from a local development environment to a cloud infrastructure—specifically Kaggle's free-tier GPU environment featuring NVIDIA T4 or P100 accelerators—requires a highly structured synchronization workflow utilizing the Kaggle Command Line Interface (CLI) and API.⁴

Local Project Directory Architecture

The foundational step in ensuring seamless integration between the local IDE and the Kaggle cloud environment involves establishing a strict, modular directory hierarchy. Monolithic scripts are prone to failure and are difficult to debug in remote instances. The codebase must be segregated into functional components, ensuring that dependencies and relative paths map correctly when the entire repository is containerized and transferred to the remote server.

Directory / File Path	Functional Description and Component Logic
/src/data_loader.py	Contains the Google Earth Engine API streaming logic, custom PyTorch Dataset classes, and the radiometric normalization algorithms.
/src/model_swinir.py	Houses the structural definitions for the Swin Transformer architecture, including the residual blocks and pixel-shuffle layers.
/src/loss_functions.py	Defines the composite multi-term loss functions, including perceptual loss models and downsampling consistency guardrails.
/src/utils_memory.py	Contains the Rasterio windowing logic, NumPy stride trick configurations, and the 2D spline blending algorithms for patch merging.
/notebooks/main_training.ipynb	The primary execution entry point. This notebook imports the modules from /src/, initializes the training loop, and logs metrics.
/configs/hyperparams.yaml	A configuration manifest defining learning rates, batch sizes, patch dimensions, and API authentication paths.
/requirements.txt	A strict manifest of package dependencies (e.g., rasterio, earthengine-api, timm) to ensure environmental parity in the cloud.

/kernel-metadata.json	The Kaggle API instruction file dictating the remote hardware provisioning, execution state, and dataset linking parameters.
-----------------------	--

The Kaggle API Deployment Logic

To automate the deployment of local code to Kaggle, the developer must employ the kaggle kernels push methodology.⁴ Attempting to manually upload scripts via the browser interface breaks the continuous integration cycle. The workflow logic operates through the kernel-metadata.json file, which acts as an instruction set for the Kaggle execution engine. The deployment sequence begins with local authentication. The Kaggle API token (a kaggle.json file generated from the user's account settings) must be securely stored in the local environment, typically under the ~/.kaggle/ directory or injected via environment variables.⁶ Once authenticated, the developer navigates to the project directory and initializes the kernel metadata using the kaggle kernels init command, which generates a template JSON file.⁴ This JSON file must be explicitly modified to govern the remote execution state. The metadata parameters define not only where the code resides but also the specific hardware required to run it.

JSON Metadata Key	Required Value and Logical Purpose
"id"	Formatted as "username/kernel-title". Serves as the unique registry identifier on the Kaggle platform.
"title"	The public-facing or private title of the training pipeline instance.
"code_file"	The relative local path pointing to the primary execution notebook (e.g., "notebooks/main_training.ipynb").
"language"	Must be set to "python" to ensure the correct kernel runtime is initialized.
"kernel_type"	Must be set to "notebook" or "script" depending on the base execution file type.

"is_private"	Set to true during development to protect proprietary architectural modifications.
"enable_gpu"	Critically, must be set to true to provision the remote hardware accelerator (NVIDIA T4 or P100). ⁴
"enable_internet"	Set to true to allow the installation of custom dependencies and permit outgoing API calls to Google Earth Engine.

If the local project is heavily modularized across multiple Python files within the /src/ directory, Kaggle handles this complexity via "Utility Scripts" or by packaging the repository. When the developer executes the `kaggle kernels push -p <FOLDER_PATH>` command, the Kaggle API packages the designated files, uploads them to the platform, provisions the defined GPU hardware, and initiates the training run without requiring any browser interaction.⁴ Output logs, saved model checkpoints (typically .pth files), and metric visualizations are subsequently pulled back to the local IDE using the `kaggle kernels output` command.⁸

Data Sourcing, Ingestion, and Radiometric Normalization

Training a deep learning super-resolution model fundamentally requires highly curated, spatially and temporally aligned pairs of low-resolution and high-resolution imagery. The WorldStrat dataset fulfills this precise requirement, providing a globally stratified corpus of high-resolution Airbus SPOT 6/7 imagery (1.5m/pixel) temporally matched with low-resolution Sentinel-2 imagery (10m/pixel).¹

The dataset encompasses nearly 10,000 square kilometers of unique global locations. Crucially, to ensure robust model generalization, these locations are not limited to pristine urban centers. They are stratified to represent all types of land-use across the world, including agriculture, ice caps, and forests. Furthermore, the dataset is enriched with locations typically under-represented in standard machine learning datasets, including sites of humanitarian interest, illegal mining operations, and settlements of persons at risk.² Training on such diverse geographical features ensures that the resulting super-resolution model can accurately reconstruct complex, chaotic environments rather than just highly structured city grids.

Ingestion Logic via Google Earth Engine Streaming

The conventional approach to deep learning involves downloading raw datasets to a local hard drive or cloud storage bucket. However, raw geospatial datasets often span terabytes, making this approach computationally prohibitive, slow, and highly inefficient for constrained cloud environments like Kaggle.¹⁰ To bypass massive local downloads and storage bottlenecks, the architecture mandates the use of the Google Earth Engine (GEE) Python API. The pipeline must

dynamically stream and tile image patches directly into the PyTorch DataLoader's memory buffer in real-time.¹

While historical implementations utilized GEE's export functions (sending data to Google Drive or Cloud Storage), this introduces severe latency and scalability issues ("User memory limit exceeded" errors).¹¹ The modern, highly optimized algorithmic approach utilizes the `ee.data.computePixels` method.¹¹ This method executes arbitrary mathematical computations on the Google servers and streams the resultant pixel data directly back to the active Python environment as a structured payload.¹²

The algorithmic flow for streaming data requires precise geographic coordinate mapping. The data loader establishes a grid of bounding boxes corresponding to the regions of interest extracted from the WorldStrat metadata. For each training step, the data loader worker constructs a dictionary payload defining the spatial query and submits it to the GEE API.

API Payload Parameter	Configuration and Algorithmic Purpose
expression	The underlying GEE Image object reference (e.g., the specific Sentinel-2 surface reflectance asset).
fileFormat	Must be set to 'NUMPY_NDARRAY'. ¹² This instructs the API to bypass compressed image formats (like PNG or GeoTIFF) and return a raw, structured array ready for tensor conversion. ¹²
bandsIds	Specifies the required spectral bands. For visible spectrum SR, this targets `` (Red, Green, Blue) from Sentinel-2. ¹²
grid	A nested dictionary defining the request geometry. It includes dimensions (e.g., 256x256 patches), the crsCode (e.g., 'EPSG:4326'), and the affineTransform mapping coordinates to pixels. ¹²

By executing these requests asynchronously within a multi-threaded data loader, the pipeline ensures that the GPU is never starved for data, maintaining a high utilization rate without requiring terabytes of persistent disk storage.

Radiometric Normalization: 16-bit to 8-bit Algorithmic Scaling

Raw multispectral satellite imagery is captured natively in 12-bit or 16-bit dynamic ranges. This high bit-depth is necessary to preserve subtle variations in surface reflectance across extreme

lighting conditions, from highly reflective snowcaps to deep, light-absorbing water bodies. However, deep learning architectures are predominantly optimized to process 8-bit data that

has been scaled into floating-point tensors normalized between $[-1, 1]$.

A naive approach to normalization involves dividing the pixel array by the maximum theoretical 16-bit value (65,535). This method fails catastrophically in practice. Genuine surface reflectance values rarely utilize the upper echelons of the 16-bit spectrum; the vast majority of pixel data clusters in the lower registers. Dividing by 65,535 heavily skews the distribution toward zero, resulting in a practically black image devoid of recoverable gradient information.

The required logic is a robust algorithmic normalization known as **Percentile Clipping**.¹⁶ This mathematical isolation is executed on the fly during the data loading process, stabilizing the statistical distribution of the imagery before it enters the neural network.

The algorithm flows as follows: First, for an incoming raw 16-bit array patch, the pipeline

calculates the 2nd percentile (P_2) and the 98th percentile (P_{98}) pixel values across each specific spectral band independently.¹⁷ This dynamic calculation identifies and isolates extreme anomalies, such as highly reflective cloud artifacts, sensor errors, or deep shadow noise, without relying on hardcoded thresholds that might fail across different geographic locations. Second, a threshold enforcement function is applied to the array. Any pixel value falling below

the P_2 threshold is clamped and set to P_2 . Conversely, any pixel value exceeding the P_{98} threshold is clamped to P_{98} .

Third, the algorithm applies Min-Max scaling to map the clamped array into a standard 8-bit spatial range of $[0, 255]$ using the following transformation logic:

$$I_{8bit} = \left(\frac{I_{16bit} - P_2}{P_{98} - P_2} \right) \times 255$$

Finally, this stabilized 8-bit array is converted into a 32-bit floating-point PyTorch tensor and scaled to $[-1, 1]$ by dividing by 255.0. This ensures that the neural network receives standardized, well-distributed data, allowing the gradient descent optimization to converge smoothly.

Advanced Model Architecture: Swin Transformer (SwinIR)

To bridge the 10m to 1.5m resolution gap effectively, standard Convolutional Neural Networks (CNNs) like SRResNet or EDSR are increasingly considered insufficient.¹ While CNNs excel at extracting localized features via their sliding receptive fields, they fundamentally struggle to model long-range dependencies. In satellite imagery, geographically distant context—such as the structural continuity of a highway network, the geometric patterns of agricultural plots, or the meandering path of a riverbed—heavily informs the localized structural prediction.¹⁸ A localized convolution kernel cannot "see" the broader context required to reconstruct these features accurately.

Therefore, the architectural blueprint mandates the implementation of **SwinIR** (Image Restoration Using Shifted Window Transformer). SwinIR leverages hierarchical Vision Transformers (ViTs) to achieve state-of-the-art super-resolution metrics, consistently outperforming older GAN and CNN paradigms.¹⁸

The SwinIR model is constructed within the PyTorch framework and is logically partitioned into three distinct, interconnected functional modules: Shallow Feature Extraction, Deep Feature Extraction, and High-Quality Image Reconstruction.²¹

1. Shallow Feature Extraction Module

The primary objective of the shallow extraction module is to map the low-resolution, 3-channel input space into a high-dimensional feature embedding while preserving critical low-frequency spatial information.²¹ Transformers process data as abstract tokens, which can destroy the inherent spatial grid of an image. The shallow module prevents this by anchoring the input.

The logic requires implementing a single 3×3 Convolutional Layer. An input tensor of shape (B, C_{in}, H, W) —where B is batch size, C_{in} is the 3 RGB channels, and H, W are the spatial dimensions—is projected to (B, C_{feat}, H, W) . C_{feat} represents the network's embedding dimension, typically set to 180 in advanced configurations. This shallow feature tensor is explicitly preserved in memory. It bypasses the entire transformer backbone and is transmitted directly via a long global skip connection to the final reconstruction module.²¹ This guarantees that the original color space and low-frequency structures are not lost during the deep transformative processes.

2. Deep Feature Extraction Module

This module serves as the core computational engine of the architecture. It relies on a cascading sequence of Residual Swin Transformer Blocks (RSTBs) to perform the heavy lifting of high-frequency feature generation.¹⁹

Deep Extraction Component	Architectural Function and Configuration
Residual Swin Transformer Block (RSTB)	The primary macroscopic building block. An RSTB encapsulates L sequential Swin Transformer Layers. Crucially, it terminates with a 3×3 Convolutional Layer and a residual connection that adds the block's input tensor directly back to its output. ²² The terminal convolution introduces inductive bias, anchoring the transformer's

	abstract representations back to physical spatial realities. ²⁵
Swin Transformer Layer (STL)	The microscopic attention mechanism. Unlike standard Vision Transformers that compute global self-attention across the entire image (which scales quadratically with image resolution $O(N^2)$ and causes OOM crashes on large satellite scenes), the STL computes localized attention. ²⁶
Window Multi-Head Self-Attention (W-MSA)	The spatial feature map is partitioned into non-overlapping local windows of size $M \times$ (typically $8 \times$). Self-attention is computed exclusively within the confines of each independent window, reducing computational complexity to a linear scale relative to image size. ²²
Shifted Window Mechanism (SW-MSA)	If attention is only computed within isolated windows, the network cannot understand relationships across window boundaries. To solve this, the subsequent STL utilizes a Shifted Window approach. The grid is displaced diagonally by $\lfloor \frac{M}{2} \rfloor$ pixels. ²⁴ This forces the attention mechanism to capture dependencies across the boundaries of the previous layer's windows, facilitating seamless contextual awareness across the entire satellite image.

Each attention operation within these layers is encapsulated with Layer Normalization to stabilize training and is followed by a Multi-Layer Perceptron (MLP) containing Gaussian Error Linear Unit (GELU) activations to introduce necessary non-linearity.²⁷

3. High-Quality Image Reconstruction Module

Once the deep features are extracted, they are added to the preserved shallow features. The combined, highly enriched tensor must now be upscaled spatially by the target factor (e.g., 4x or 8x) to produce the final synthetic commercial-grade image.²¹

Historically, upscaling was achieved using transposed convolutions (deconvolutions), which frequently resulted in checkerboard artifacts. The modern architectural standard requires the

implementation of a **PixelShuffle** layer, mathematically defined as Sub-Pixel Convolution.²⁸

The algorithmic flow for the upsampler begins with a 3×3 convolutional expansion layer. This layer expands the channel dimension of the deep feature tensor by the square of the desired upscale factor r . For an upscale factor of $r = 4$, the channels are multiplied by 16. The tensor shape transitions from (B, C, H, W) to $(B, C \cdot r^2, H, W)$. The PixelShuffle operation then deterministically rearranges this massive depth dimension into spatial dimensions. It effectively takes a $1 \times 1 \times 16$ column of pixels and unwraps it into a $4 \times 4 \times 1$ spatial patch. The resulting high-resolution tensor assumes the shape $(B, C, r \cdot H, r \cdot W)$.³⁰ Finally, a standard convolutional layer collapses the abstract embedding dimension back to the 3 standard RGB color channels, yielding the final super-resolved output.

The Composite Loss Function and Hallucination Guardrails

Optimizing a neural network to produce commercial-grade satellite imagery cannot rely on a single loss metric. A multi-objective loss function ensemble is required to balance pixel-perfect fidelity with structural realism, while explicitly penalizing geospatial fabrication.¹ The total optimization objective is defined by the following weighted formulation:

$$\mathcal{L}_{Total} = \lambda_1 \mathcal{L}_{Pixel} + \lambda_2 \mathcal{L}_{Perceptual} + \lambda_3 \mathcal{L}_{Adversarial} + \lambda_4 \mathcal{L}_{Downsample}$$

Loss Component	Objective and Mathematical Rationale
Pixel Loss ($\mathcal{L}_{Pixel}$)	Enforces baseline color and structural matching. The architecture must utilize L_1 (Mean Absolute Error) rather than L_2 (Mean Squared Error). L_2 heavily penalizes large errors, causing the network to average out uncertainty, which results in systemic, smoothed blurring. L_1 converges to sharper median estimations, preserving high-frequency edges. ³¹

Perceptual Loss ($\mathcal{L}_{Perceptual}$)	Optimizes how the image "looks" to human perception. This is achieved by extracting high-level feature maps from a pre-trained VGG-19 classification network (or using LPIPS). The loss is calculated as the Euclidean distance between the VGG abstract representations of the predicted HR image and the ground-truth HR image, enforcing texture matching rather than just exact pixel alignment. ³²
Adversarial Loss ($\mathcal{L}_{Adversarial}$)	Pushes visual fidelity to commercial standards by integrating a Relativistic Average GAN (RaGAN) discriminator. ¹⁸ Rather than predicting an absolute "real or fake" value, RaGAN predicts the probability that the ground-truth real image is <i>more authentic</i> than the generated image. This continuous gradient forces the generator to produce highly realistic edge textures, such as sharp building footprints and defined road lines.

The Hallucination Guardrail Mechanism

While the aforementioned metrics evaluate the presence of noise and structural correlation, they do not inherently flag localized generative failures. In natural image enhancement (e.g., upscaling a photograph of a face), hallucinating synthetic skin texture is acceptable. In remote sensing, hallucinating a non-existent building, or converting a river into a road, constitutes a catastrophic system failure.¹

To prevent this, an advanced pipeline must institute an explicit logical guardrail against topological inventions. The architectural solution is the implementation of **Downsampling**

Consistency Loss ($\mathcal{L}_{Downsample}$), also referred to as cycle consistency.³⁴

The fundamental physical law of super-resolution dictates that an ideal high-resolution image, if subjected to the sensor degradation of the original satellite, must perfectly yield the exact low-resolution observation. To implement this, the pipeline constructs a mathematical

degradation operator A that simulates the Point Spread Function (PSF), atmospheric blurring, and pixel binning of the Sentinel-2 sensor.³⁴

During the training and evaluation loop, the generated SR output is passed through the

degradation operator A . A fidelity check is then performed by computing the Mean Absolute

Error (MAE) between this artificially degraded SR image and the original Sentinel-2 LR input tensor.

$$\mathcal{L}_{\text{Downsample}} = \|\text{Degradation}(X_{\text{Generated_HR}}) - X_{\text{Original_LR}}\|_2^2$$

If the model hallucinates macro-structures—such as fabricating an urban block where a forest exists—those structures will persist through the downsampling operator. The resulting degraded image will drastically mismatch the original low-resolution input, causing a massive

spike in the $\mathcal{L}_{\text{Downsample}}$ loss. The backpropagation of this penalty actively forces the network's weights to suppress the invention of ungrounded features, mathematically guaranteeing geographical fidelity.³⁴

Out-of-Core Memory Management and Geospatial Tiling Strategy

Training and executing inference on high-resolution geospatial models within hardware-constrained environments like Kaggle presents severe memory bottlenecks. Standard satellite acquisitions, such as Sentinel-2 granules, encapsulate thousands of square kilometers within a single multi-gigabyte array. Attempting to load an entire full-resolution satellite array into GPU VRAM (typically limited to 16GB on a Kaggle free-tier T4) triggers immediate out-of-memory (OOM) fatal crashes, terminating the kernel.¹

To process vast geographical extents, the architecture must utilize a specialized spatial slicing strategy known as "windowing" or "tiling." This approach breaks the massive image into manageable blocks, processes them sequentially through the neural network, and stitches them back together into a continuous canvas.³⁷

Geospatial Tiling Logic via Rasterio

Rasterio is the optimal Python library for interfacing with geospatial data formats (like GeoTIFFs) because it supports out-of-core processing. Rather than eagerly loading an entire massive array into system RAM, Rasterio permits targeted, windowed reading and writing.³⁷ The algorithmic sequence for windowed extraction begins by initializing the dataset metadata. The script opens the source file and extracts the spatial metadata, including the Coordinate Reference System (crs), the affine transform matrix (transform), and the master dimensions (width, height). Programmatically, a sliding window generator computes bounding boxes that traverse the master image grid, defining patches of a specified dimension (e.g., 256×256 pixels).

The extraction utilizes the `rasterio.windows.Window(col_off, row_off, width, height)` function to selectively pull specific chunks of memory from the disk directly to the CPU, bypassing full file ingestion.³⁹ Critically, when extracting a tile, its geospatial footprint must be meticulously preserved. The script computes a new affine transform specifically for the extracted window using the `rasterio.windows.transform(window, dataset.transform)` function.³⁹ This guarantees that when the super-resolved patch is eventually written back to disk, it occupies the exact

geographical coordinates in the real world as the low-resolution input, preventing spatial drifting.⁴¹

Overlapping Patches and NumPy Stride Tricks

To maintain contextual awareness across tile boundaries, the extracted patches must overlap. If patches are extracted and processed continuously without overlap, the neural network suffers from a restricted, incomplete receptive field near the edges of the patch. Because the network lacks surrounding context, it predicts poorly at the margins, leading to distinct, grid-like seamlines and artificial boundaries in the final stitched output.⁴²

Generating overlapping patches using standard Python for loops is computationally slow and duplicates memory unnecessarily. To extract overlapping patches instantly and efficiently, the pipeline relies on NumPy's stride mechanics. Instead of copying data, a custom view of the memory is generated using `numpy.lib.stride_tricks.as_strided`.⁴⁴

Strides dictate the number of bytes the CPU must jump in memory to reach the next element in a multi-dimensional array. By defining a custom step size (for instance, a window of 256 pixels with a step size of 128 yields a 50% overlap), `as_strided` "patchifies" the master array virtually instantaneously.⁴⁴ This creates a high-performance, read-only view of overlapping tiles that can be fed directly into the deep learning DataLoader without duplicating the underlying data in RAM.

Seamless Patch Blending Algorithm

Reconstructing the super-resolved tiles back into a massive, continuous high-resolution canvas presents the final mathematical challenge: pixels located in the overlapping regions will possess multiple different predicted values derived from different adjacent tiles. A naive averaging mechanism applied to these overlaps will inadvertently blur the high-frequency details the network worked so hard to generate.

The advanced architectural solution to this problem is **2D Spline Window Blending**, originally conceptualized to eliminate edge effects in satellite segmentation tasks.⁴²

The blending algorithm operates on the premise that a neural network's prediction is most confident and accurate at the dead center of a patch—where it has 360-degree surrounding spatial context—and becomes increasingly unreliable toward the extreme edges where context is truncated.

The logic dictates generating a 2D spline weight matrix (typically a Hanning window or a Squared Spline window) that matches the exact dimensions of the output patch.⁴³ This matrix

functions as a spatial confidence map. The numerical weights approach **1.0** at the exact center of the patch and smoothly, non-linearly decay to **0.0** at the extreme edges.⁴⁶

Before inference begins, the script allocates a zero-filled, high-resolution master NumPy array to hold the final image predictions, alongside an identically shaped array to accumulate the weights. As the SwinIR model predicts an upscaled high-resolution patch, the raw output tensor is mathematically multiplied by the 2D spline weight matrix.

This weighted prediction patch is then added to its corresponding spatial location in the master

prediction canvas. Simultaneously, the 2D spline weight matrix itself is added to the corresponding location in the master weight canvas. Once all patches across the entire geographic region are processed, the master prediction canvas is divided by the master weight canvas on a pixel-by-pixel basis.

This algorithm seamlessly merges predictions without blurring. Because the center of a patch carries the highest numerical weight, it naturally overrules the less reliable predictions made at the edges of adjacent overlapping patches. The mathematical division normalizes the accumulation, resulting in a continuous, visually flawless image entirely devoid of jagged grid-line artifacts.⁴²

Geospatial Accuracy and Quantitative Evaluation Metrics

The criteria for evaluating a commercial-grade satellite imagery super-resolution pipeline differ starkly from natural, consumer-grade image enhancement.¹ The architecture mandates strict mathematical evaluation systems to objectively quantify the fidelity of the reconstruction. The fundamental baseline for any super-resolution comparison is standard **Bicubic Interpolation**.¹ Bicubic scaling uses a mathematical polynomial to guess the values of new pixels based on a 4×4 neighborhood of surrounding pixels. It produces smooth, artificially blurred upscaling without introducing new high-frequency details. To validate the deep learning model, the SwinIR outputs must drastically exceed the Bicubic baseline across two primary objective metrics: Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index Measure (SSIM).¹

Evaluation Metric	Mathematical Formulation and Application
Peak Signal-to-Noise Ratio (PSNR)	Measures the ratio between the maximum possible pixel intensity and the corruption noise (the error between the predicted image and the ground truth commercial image).¹ The algorithm first computes the Mean Squared Error (MSE) between the generated HR image (I_{SR}) and the ground truth HR image (I_{HR}). $MSE = \frac{1}{H \cdot W} \sum_{i=1}^H \sum_{j=1}^W (I_{SR}(i,j) - I_{HR}(i,j))^2$

	The PSNR is then derived via a logarithmic scale: $PSNR = 10 \cdot \log_{10} \left(\frac{MAX^2}{MSE} \right)$ A higher PSNR (measured in decibels, dB) implies that the predicted pixel values align more closely with the ground truth.
Structural Similarity (SSIM)	While PSNR measures absolute pixel difference, it fails to account for human visual perception. SSIM models how the human visual system perceives image degradation by assessing three independent components: luminance, contrast, and structural cohesion.¹ The metric computes local means (μ), variances (σ^2), and covariance (σ_{xy}) for both images across sliding geographical windows. $SSIM = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$ An SSIM score approaching 1.0 dictates that the macro-structural geometries (e.g., the right angles of building roofs, the linearity of roads) have been accurately reconstructed without distortion.

By tracking these metrics throughout the training epochs on a holdout validation set from the WorldStrat dataset, the developer can mathematically prove that the neural network is actively learning to reconstruct high-frequency geographical data, rather than merely smoothing pixels as the Bicubic baseline does.

Processing and User Interface Blueprint: Interactive Visualization

The culmination of an industry-grade machine learning pipeline extends beyond back-end mathematical processing and loss optimization; it demands a robust, intuitive front-end application to visualize and compare the capabilities of the model effectively.¹ Demonstrating inference on a "Mystery Location"—a geographic coordinate completely unseen by the network during training—proves the model's capacity for true generalization.

To achieve this, the architectural blueprint utilizes **Streamlit**, a highly efficient Python-based framework designed specifically for deploying data science applications without requiring complex HTML/CSS or JavaScript frontend development.⁴⁹

The user interface must allow an end-user to fluidly compare the blurry, 10m/pixel Sentinel-2 baseline against the sharp, 1.5m/pixel synthetic commercial-grade output generated by the SwinIR model. This is best achieved via an interactive, draggable image comparison slider.¹

The structural logic for building this application follows a distinct state-driven sequence: First, the environment is initialized. The Streamlit application (app.py) is configured utilizing the `st.set_page_config()` method to establish a wide layout format, ensuring that high-resolution satellite imagery is rendered properly without UI clipping.⁴⁹

Second, the data ingestion pipeline is established on the front end. The application implements an interactive component (such as `st.file_uploader` or a pre-populated geographic dropdown menu) to allow users to load the low-resolution "Mystery Location" GeoTIFF into the application's memory state.

Third, the inference execution state is triggered. When the user initiates the upscaling process, a visual progress mechanism (`st.spinner`) engages, locking the UI while the backend executes the model. The backend script loads the pre-trained SwinIR PyTorch checkpoint, applies the Rasterio sliding-window extraction across the loaded image, executes the model over the tiled patches, and merges them using the 2D Spline Window Blending algorithm to construct the final high-resolution array.

To ensure parity and fairness in the visual comparison, the original low-resolution image must be algorithmically upscaled using standard Bicubic interpolation to match the exact physical spatial dimensions of the super-resolved output. Attempting to overlay a small image directly on top of a large image would break the visualization component.

Finally, the visualization leverages an external component wrapper, specifically `streamlit_image_comparison` (which utilizes the Knightlab JuxtaposeJS library under the hood).⁴⁹ The logic dictates feeding the upscaled Bicubic array as `img1` (the "Before" state) and the SwinIR array as `img2` (the "After" state) into the `image_comparison()` function. Parameters such as `label1="Sentinel-2 (Bicubic Baseline)"` and `label2="SwinIR (Synthetic Commercial)"` are injected to provide clear context.⁴⁹

This component renders an interactive overlay interface directly in the browser. The user can drag a vertical slider horizontally across the canvas, dynamically exposing the enhanced high-frequency details. This slider effectively demonstrates the model's ability to pull latent information out of the blurry input—revealing newly visible lane markings on highways, distinct edges of complex agricultural plots, and highly structured building footprints—proving the real-world value of the super-resolution pipeline.⁴⁹

Conclusion

Developing an industry-grade Super-Resolution pipeline capable of crossing the spatial divide between free, low-resolution satellite monitoring and costly, commercial-grade imagery transcends simple neural network implementation; it requires a holistic, systems-level architectural blueprint. By synchronizing local development environments with cloud-based hardware accelerators, developers can maintain rapid prototyping cycles without sacrificing the massive compute required for transformer training. Bypassing massive static data downloads via dynamic Google Earth Engine API streaming eradicates storage bottlenecks, while percentile-based radiometric normalization ensures the stable ingestion of well-distributed mathematical tensors.

At the core of the pipeline, the deployment of the SwinIR Vision Transformer leverages localized, shifted-window self-attention mechanisms to model both the micro-textures and the macro-geographic contexts required for accurate reconstruction. This complex architecture is governed by a multi-objective loss ensemble, critically anchored by downsampling consistency checks that serve as unyielding mathematical guardrails against the fabrication of non-existent geospatial features. Furthermore, strictly enforcing out-of-core memory management via Rasterio tiling, NumPy memory striding, and 2D spline blending prevents catastrophic hardware failures and eliminates seamline artifacts across massive landscapes. Ultimately, integrating these complex back-end operations with a fluid, interactive Streamlit comparison interface yields a truly advanced, highly capable solution, empowering researchers to extract unprecedented value from global Earth Observation systems.

Works cited

1. 697e391675772_Ascent-ML-Track.pdf
2. [2207.06418] Open High-Resolution Satellite Imagery: The WorldStrat Dataset -- With Application to Super-Resolution - arXiv, accessed May 26, 2026, <https://arxiv.org/abs/2207.06418>
3. Open High-Resolution Satellite Imagery: The WorldStrat Dataset – With Application to Super-Resolution - arXiv, accessed May 26, 2026, <https://arxiv.org/html/2207.06418v2>
4. kaggle-cli/docs/kernels.md at main - GitHub, accessed May 26, 2026, <https://github.com/Kaggle/kaggle-cli/blob/main/docs/kernels.md>
5. Public API - Kaggle, accessed May 26, 2026, <https://www.kaggle.com/docs/api>
6. Kaggle/kagglehub: Python library to access Kaggle resources - GitHub, accessed May 26, 2026, <https://github.com/Kaggle/kagglehub>
7. Scheduling notebook execution with cron and Kaggle API, accessed May 26, 2026, <https://www.kaggle.com/discussions/product-feedback/371090>
8. Kaggle Public API - kernels, accessed May 26, 2026, <https://www.kaggle.com/code/k1gaggle/kaggle-public-api-kernels>
9. WorldStrat - Kaggle, accessed May 26, 2026, <https://www.kaggle.com/datasets/jucor1/worldstrat>
10. Major TOM: Expandable Datasets for Earth Observation - arXiv, accessed May 26,

- 2026, <https://arxiv.org/html/2402.12095v2>
11. Pixels to the people! - Google Earth and Earth Engine - Medium, accessed May 26, 2026, <https://medium.com/google-earth/pixels-to-the-people-2d3c14a46da6>
 12. ee.data.computePixels (Python only) | Google Earth Engine, accessed May 26, 2026, <https://developers.google.com/earth-engine/apidocs/ee-data-computepixels>
 13. Earth Engine Data Converters · GitHub, accessed May 26, 2026, <https://gist.github.com/jdbcode/cc1db9e9555bf988e4ac3ebca5fa467b>
 14. Super-Resolution of Sentinel-2 Satellite Images: A Comparison of Different Interpolation Methods for Spatial Knowledge Extraction - MDPI, accessed May 26, 2026, <https://www.mdpi.com/2504-4990/8/1/14>
 15. Earth Engine Data Converters - Google for Developers, accessed May 26, 2026, <https://developers.google.com/earth-engine/tutorials/community/data-converters>
 16. Investigating Blind Image Super-Resolution of Sentinel-2 Satellite Data and Its Applications, accessed May 26, 2026, https://elib.dlr.de/219126/1/Ron_Bachelor_Thesis.pdf
 17. Strategies for converting 16-bit multispectral satellite imagery to 8-bit - GIS StackExchange, accessed May 26, 2026, <https://gis.stackexchange.com/questions/498228/strategies-for-converting-16-bit-multispectral-satellite-imagery-to-8-bit>
 18. Structure-preserving super-resolution of retinal fundus images via a dual-transformer residual network - PMC, accessed May 26, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12900677/>
 19. SwinIR: Image Restoration Using Swin Transformer (official repository) - GitHub, accessed May 26, 2026, <https://github.com/jingyunliang/swinir>
 20. SwinIR: Image Restoration Using Swin Transformer - CVF Open Access, accessed May 26, 2026, https://openaccess.thecvf.com/content/ICCV2021W/AIM/papers/Liang_SwinIR_Image_Restoration_Using_Swin_Transformer_ICCVW_2021_paper.pdf
 21. Stunning Art with CLIP Guided Diffusion+SwinIR - Kaggle, accessed May 26, 2026, <https://www.kaggle.com/code/sreevishnudamodaran/stunning-art-with-clip-guided-diffusion-swinir>
 22. Cross-Modality Guided Super-Resolution for Weak-Signal Fluorescence Imaging via a Multi-Channel SwinIR Framework - MDPI, accessed May 26, 2026, <https://www.mdpi.com/2079-9292/15/1/204>
 23. C-SwinIR: Dental CT super-resolution reconstruction fused with SwinIR - ResearchGate, accessed May 26, 2026, https://www.researchgate.net/publication/374178704_C-SwinIR_Dental_CT_super-resolution_reconstruction_fused_with_SwinIR
 24. SwinIR (Shifted windows Image Restoration) model for SISR (Single Image Super-Resolution) task using PyTorch - GitHub, accessed May 26, 2026, <https://github.com/ash1ra/SwinIR>
 25. Unified Efficient Hierarchical Swin Attention Networks for Low-Dose PET and CT Denoising, accessed May 26, 2026, <https://arxiv.org/html/2509.06591v7>
 26. Swin Transformer in Depth: Architecture and PyTorch Implementation | by Övül

- Arslan, accessed May 26, 2026,
<https://medium.com/@ovularslan/swin-transformer-in-depth-architecture-and-pytorch-implementation-a11e44d65aef>
27. Swin Transformer Explained: Architecture and PyTorch Implementation - Aman Arora's Blog, accessed May 26, 2026,
<https://amaarora.github.io/posts/2022-07-04-swintransformerv1.html>
 28. Swin2SR - Hugging Face, accessed May 26, 2026,
https://huggingface.co/docs/transformers/model_doc/swin2sr
 29. A Lightweight Image Super-Resolution Transformer Trained on Low-Resolution Images Only - arXiv, accessed May 26, 2026, <https://arxiv.org/html/2503.23265v1>
 30. A Transformer-Based Model for Super-Resolution of Anime Image - PMC, accessed May 26, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC9657210/>
 31. Transformer for Single Image Super-Resolution - CVF Open Access, accessed May 26, 2026,
https://openaccess.thecvf.com/content/CVPR2022W/NTIRE/papers/Lu_Transformer_for_Single_Image_Super-Resolution_CVPRW_2022_paper.pdf
 32. Investigating Loss Functions for Extreme Super-Resolution - CVF Open Access, accessed May 26, 2026,
https://openaccess.thecvf.com/content_CVPRW_2020/papers/w31/Jo_Investigating_Loss_Functions_for_Extreme_Super-Resolution_CVPRW_2020_paper.pdf
 33. Deep Learning for Satellite Image Super-Resolution: Mainstream Methods in the State of the Art - Digital Sense, accessed May 26, 2026,
<https://www.digitalsense.ai/blog/deep-learning-for-satellite-image-super-resolution-mainstream-methods-in-the-state-of-the-art>
 34. arXiv:2211.13524v1 [cs.CV] 24 Nov 2022, accessed May 26, 2026,
<https://arxiv.org/pdf/2211.13524>
 35. Attention-Enhanced GAN for Spatial-Spectral Fusion and Chlorophyll-a Inversion in Chen Lake, China - MDPI, accessed May 26, 2026,
<https://www.mdpi.com/1424-8220/26/7/2107>
 36. Overcoming Hallucinations in Satellite Imagery Super-Resolution: A High-Fidelity ESRGAN Approach - ResearchGate, accessed May 26, 2026,
https://www.researchgate.net/publication/404775742_Overcoming_Hallucinations_in_Satellite_Imagery_Super-Resolution_A_High-Fidelity_ESRGAN_Approach
 37. Windowed reading and writing — rasterio 1.4.4 documentation, accessed May 26, 2026, <https://rasterio.readthedocs.io/en/stable/topics/windowed-rw.html>
 38. How to efficiently create millions of overlapping raster tiles - Nick Wright - GitHub Pages, accessed May 26, 2026,
<https://dpird-dma.github.io/blog/How-to-efficiently-create-millions-of-overlapping-raster-tiles/>
 39. rasterio.windows module - Read the Docs, accessed May 26, 2026,
<https://rasterio.readthedocs.io/en/stable/api/rasterio.windows.html>
 40. How to split multiband image into image tiles using Rasterio? - GIS StackExchange, accessed May 26, 2026,
<https://gis.stackexchange.com/questions/285499/how-to-split-multiband-image-into-image-tiles-using-rasterio>

41. Unexpected result using rasterio windowed writing - Access window out of range in RasterIO() - GIS StackExchange, accessed May 26, 2026, <https://gis.stackexchange.com/questions/349183/unexpected-result-using-rasterio-windowed-writing-access-window-out-of-range-i>
42. Using a U-Net for image segmentation, blending predicted patches smoothly is a must to please the human eye. - GitHub, accessed May 26, 2026, <https://github.com/Vooban/Smoothly-Blend-Image-Patches>
43. Smoothly-Blend-Image-Patches/README.md at master - GitHub, accessed May 26, 2026, <https://github.com/Vooban/Smoothly-Blend-Image-Patches/blob/master/README.md>
44. Fast Way to slice image into overlapping patches and merge patches to image - Stack Overflow, accessed May 26, 2026, <https://stackoverflow.com/questions/16774148/fast-way-to-slice-image-into-overlapping-patches-and-merge-patches-to-image>
45. Is there any way to patchify the array with overlap with numpy methods? - Stack Overflow, accessed May 26, 2026, <https://stackoverflow.com/questions/53137528/is-there-any-way-to-patchify-the-array-with-overlap-with-numpy-methods>
46. Smoothly-Blend-Image-Patches/smooth_tiled_predictions.py at master - GitHub, accessed May 26, 2026, https://github.com/Vooban/Smoothly-Blend-Image-Patches/blob/master/smooth_tiled_predictions.py
47. 229 - Smooth blending of patches for semantic segmentation of large images (using U-Net), accessed May 26, 2026, <https://www.youtube.com/watch?v=HrGn4uFrMOM>
48. Beyond Visual Fidelity: Benchmarking Super-Resolution Models for Large-Scale Remote Sensing Imagery via Downstream Task Integration - arXiv, accessed May 26, 2026, <https://arxiv.org/html/2605.00310v1>
49. Image comparison slider component for Streamlit - GitHub, accessed May 26, 2026, <https://github.com/fcakyon/streamlit-image-comparison>
50. Support for sliding image comparison · Issue #1382 · gradio-app/gradio - GitHub, accessed May 26, 2026, <https://github.com/gradio-app/gradio/issues/1382>
51. streamlit-image-comparison - PyPI, accessed May 26, 2026, <https://pypi.org/project/streamlit-image-comparison/>
52. Streamlit Examples on Notebooks Hub - GitHub Pages, accessed May 26, 2026, https://polusai.github.io/notebooks-hub/user/streamlit_4-polus.html